



COMPUTER
ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURES AND ALGORITHM	SECTION:	CPE21S1
DATE PERFORMED:	23/7/2026	DATE SUBMITTED:	28/7/2026
NAME:	LOREEN JAE D. PILAPIL	INSTRUCTOR:	ENGR. QUEJADO

ACTIVITY NO. 3.1	LINKED LISTS
1. PROCEDURE OUTPUT	

Screenshot

```
1  #include <iostream>
2  #include "singly_ll.h"
3  #include "double_ll.h"
4
5
6  int main(){
7
8      //CREATION OF NODE
9      doublelist<char>* head = new_Node('C');
10
11      //CREATION OF OTHER NODES
12      doublelist<char>* second = new_Node('P');
13      doublelist<char>* third = new_Node('E');
14      doublelist<char>* fourth = new_Node('1');
15      doublelist<char>* fifth = new_Node('0');
16      doublelist<char>* last = new_Node('1');
17
18      //LINK THE NODES
19      head->next = second;
20      second->prev = head;
21      second->next = third;
22      third->prev = second;
23      third->next = fourth;
24      fourth->prev = third;
25      fourth->next = fifth;
26      fifth->prev = fourth;
27      fifth->next = last;
28      last->prev = fifth;
29      last->next = nullptr;
```

	<pre> Initial Traversal: C P E 1 0 1 Reverse Traversal: 1 0 1 E P C Insertion at the head G C P E 1 0 1 Reverse Traversal: 1 0 1 E P C G General Insertion G C P E E 1 0 1 Reverse Traversal: 1 0 1 E E P C G Delete C G P E E 1 0 1 Reverse Traversal: 1 0 1 E E P G Delete P G E E 1 0 1 Reverse Traversal: 1 0 1 E E G Final G E E 1 0 1 Reverse Traversal: 1 0 1 E E G </pre>
Discussion	This set of code creates a simple doubly linked list. It creates 6 nodes in memory using <code>doublelist<char>*</code> and assigned letters and numbers to spell out 'CPE010'. It connects the nodes by manually linking their next and prev pointers back and forth.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
-----------	------------

Traversal

```

36  template <typename T>
37  void dllTraverse(doublelist<T>* currentNode){
38      doublelist<T>* tail = nullptr;
39      //TRAVERSAL TO THE END
40      while (currentNode != NULL){
41          //prints the data
42          std::cout << currentNode->data << " ";
43          //saves the current node as the last visited node
44          tail = currentNode;
45          //move to the next node
46          currentNode = currentNode->next;
47      }
48
49
50      //REVERSE TRAVERSAL
51      std::cout << "\nReverse Traversal: " << std::endl;
52      while(tail != nullptr){
53          std::cout << tail->data << " ";
54          tail = tail->prev;
55      }
56  }

```

Insertion at head

```

template <typename T>
void dllInsertHead(T newData, doublelist<T>** currentHead){

    //creates a new node
    doublelist<T>* newNode = new_Node(newData);

    //point to the current head
    newNode->next = *currentHead;

    //current head will be pointed to a new node
    (*currentHead)->prev = newNode;

    //update head pointer
    *currentHead = newNode;

}

```

Insertion at any part of the list

```
template <typename T>
void dllGenInsert(T newData, doublelist<T>* prevNode){

    if(prevNode == nullptr){
        return;
    }

    //create a new node
    doublelist<T>* newNode = new_Node(newData);

    //new node points to the node after the prevNode
    newNode->next = prevNode->next;

    //newNode previous must be pointed to the prevNode
    newNode->prev = prevNode;

    //if there is a node after the previous node
    //make it point back to the new node
    if(prevNode->next != nullptr){
        prevNode->next->prev = newNode;
    }

    //prevNode will be pointed to the newNode
    prevNode->next = newNode;
}
```

Insertion at the end	<pre>//----- //INSERT AT THE END //----- template <typename T> void dllInsertEnd(T newData, doublelist<T>* currenthead){ //Create a new node doublelist<T>* newNode = new_Node(newData); //Traverse until the last nod while(currenthead->next != nullptr){ currenthead = currenthead->next; } currenthead->next = newNode; newNode->prev = currenthead; }</pre>
Deletion at the end	<pre>135 //----- 136 //DELETION 137 //----- 138 template <typename T> 139 void dllDelete(T findData, doublelist<T>* currenthead){ 140 if(*currenthead == nullptr){ 141 return; 142 } 143 144 doublelist<T>* currentnode= *currenthead; 145 146 while(currentnode != nullptr && currentnode->data !=findData){ 147 currentnode= currentnode->next; 148 } 149 150 if (currentnode==nullptr){ 151 return; 152 } 153 if (currentnode == *currenthead){ 154 *currenthead=currentnode->next; 155 156 if (currenthead !=nullptr){ 157 (*currenthead)->prev= nullptr; 158 } 159 160 }else { 161 currentnode->prev->next= currentnode->next; 162 if(currentnode->next !=nullptr){ 163 currentnode->next->prev= currentnode->prev; 164 } 165 } 166 delete currentnode; 167 } 168 169 #endif</pre>

Table 3-2. Code for the List Operations

a	Source Code	<pre> 1 #include <iostream> 2 #include "singly_ll.h" 3 #include "double_ll.h" 4 5 6 int main(){ 7 8 //CREATION OF NODE 9 doublelist<char>* head = new_Node('C'); 10 11 //CREATION OF OTHER NODES 12 doublelist<char>* second = new_Node('P'); 13 doublelist<char>* third = new_Node('E'); 14 doublelist<char>* fourth = new_Node('1'); 15 doublelist<char>* fifth = new_Node('0'); 16 doublelist<char>* last = new_Node('1'); 17 18 //LINK THE NODES 19 head->next = second; 20 second->prev = head; 21 second->next = third; 22 third->prev = second; 23 third->next = fourth; 24 fourth->prev = third; 25 fourth->next = fifth; 26 fifth->prev = fourth; 27 fifth->next = last; 28 last->prev = fifth; 29 last->next = nullptr; 30 31 }</pre>
	Console	<pre> Initial Traversal: C P E 1 0 1 Reverse Traversal: 1 0 1 E P C</pre>

b	Source Code	<pre> 67 template <typename T> 68 void dllInsertHead(T newData, doublelist<T>** currentHead){ 69 70 //creates a new node 71 doublelist<T>* newNode = new_Node(newData); 72 73 //point to the current head 74 newNode->next = *currentHead; 75 76 //current head will be pointed to a new node 77 (*currentHead)->prev = newNode; 78 79 //update head pointer 80 *currentHead = newNode; 81 82 } std::cout << "\n Insertion at the head \n"; dllInsertHead('G', &head); dllTraverse(head); </pre>
	Console	<pre> Insertion at the head G C P E 1 0 1 Reverse Traversal: 1 0 1 E P C G </pre>

C	Source Code	<pre> 88 template <typename T> 89 void dllGenInsert(T newData, doublelist<T>* prevNode){ 90 91 if(prevNode == nullptr){ 92 return; 93 } 94 95 //create a new node 96 doublelist<T>* newNode = new_Node(newData); 97 98 //new node points to the node after the prevNode 99 newNode->next = prevNode->next; 100 101 //newNode previous must be pointed to the prevNode 102 newNode->prev = prevNode; 103 104 //if there is a node after the previous node 105 //make it point back to the new node 106 if(prevNode->next != nullptr){ 107 prevNode->next->prev = newNode; 108 } 109 110 //prevNode will be pointed to the newNode 111 prevNode->next = newNode; 112 } </pre> <pre> std::cout << "\nGeneral Insertion\n"; dllGenInsert('E', second); dllTraverse(head); </pre>
	Console	<pre> General Insertion G C P E E 1 0 1 Reverse Traversal: 1 0 1 E E P C G </pre>

d	Source Code	<pre> 136 template <typename T> 137 void dllDelete(T findData, doublelist<T>** currenthead){ 138 if(*currenthead == nullptr){ 139 return; 140 } 141 142 doublelist<T>* currentnode= *currenthead; 143 144 while(currentnode != nullptr && currentnode-> data !=findData){ 145 currentnode= currentnode->next; 146 } 147 148 if (currentnode==nullptr){ 149 return; 150 } 151 if (currentnode == *currenthead){ 152 *currenthead=currentnode->next; 153 154 if (currenthead !=nullptr){ 155 (*currenthead)->prev= nullptr; 156 } 157 158 }else { 159 currentnode->prev->next= currentnode->next; 160 if(currentnode->next !=nullptr){ 161 currentnode->next->prev= currentnode->prev; 162 } 163 } 164 delete currentnode; 165 } 166 167 #endif </pre> <pre> std::cout<<"\n Delete C \n"; dllDelete('C',&head); dllTraverse(head); </pre>
	Console	<pre> Delete C G P E E 1 0 1 Reverse Traversal: 1 0 1 E E P G </pre>

e	Source Code	<pre> 136 template <typename T> 137 void dllDelete(T findData, doublelist<T>** currenthead){ 138 if(*currenthead == nullptr){ 139 return; 140 } 141 142 doublelist<T>* currentnode= *currenthead; 143 144 while(currentnode != nullptr && currentnode-> data !=findData){ 145 currentnode= currentnode->next; 146 } 147 148 if (currentnode==nullptr){ 149 return; 150 } 151 if (currentnode == *currenthead){ 152 *currenthead=currentnode->next; 153 154 if (currenthead !=nullptr){ 155 (*currenthead)->prev= nullptr; 156 } 157 158 }else { 159 currentnode->prev->next= currentnode->next; 160 if(currentnode->next !=nullptr){ 161 currentnode->next->prev= currentnode->prev; 162 } 163 } 164 delete currentnode; 165 } 166 167 #endif std::cout<<"\n Delete P \n"; dllDelete('P',&head); dllTraverse(head); </pre>
	Console	<pre> Delete P G E E 1 0 1 Reverse Traversal: 1 0 1 E E G </pre>
f	Source Code	<pre> std::cout << "\nFinal\n"; dllTraverse(head); </pre>

	Console	<pre>Final G E E 1 0 1 Reverse Traversal: 1 0 1 E E G</pre>
--	---------	---

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots	Analysis
<pre>-- 36 template <typename T> 37 void dllTraverse(doublelist<T>* currentNode){ 38 doublelist<T>* tail = nullptr; 39 //TRAVERSAL TO THE END 40 while (currentNode != NULL){ 41 //prints the data 42 std::cout << currentNode->data << " "; 43 //saves the current node as the last visited node 44 tail = currentNode; 45 //move to the next node 46 currentNode = currentNode->next; 47 } 48 49 50 //REVERSE TRAVERSAL 51 std::cout << "\nReverse Traversal: " << std::endl; 52 while(tail != nullptr){ 53 std::cout << tail->data << " "; 54 tail = tail->prev; 55 } 56 57 58 }</pre>	traversal correctly displayed all nodes from the head to the last node by following the next pointers
<pre>template <typename T> void dllInsertHead(T newData, doublelist<T>** currentHead){ //creates a new node doublelist<T>* newNode = new_Node(newData); //point to the current head newNode->next = *currentHead; //current head will be pointed to a new node (*currentHead)->prev = newNode; //update head pointer *currentHead = newNode; }</pre>	A new node was inserted at the beginning of the list.

```

88 template <typename T>
89 void dllGenInsert(T newData, doublelist<T>* prevNode){
90
91     if(prevNode == nullptr){
92         return;
93     }
94
95     //create a new node
96     doublelist<T>* newNode = new _Node(newData);
97
98     //new node points to the node after the prevNode
99     newNode->next = prevNode->next;
100
101     //newNode previous must be pointed to the prevNode
102     newNode->prev = prevNode;
103
104     //if there is a node after the previous node
105     //make it point back to the new node
106     if(prevNode->next != nullptr){
107         prevNode->next->prev = newNode;
108     }
109
110     //prevNode will be pointed to the newNode
111     prevNode->next = newNode;
112 }

```

A new node was inserted after the specified node.

```

//-----
//INSERT AT THE END
//-----
template <typename T>
void dllInsertEnd(T newData, doublelist<T>* currenthead){

    //Create a new node
    doublelist<T>* newNode = new _Node(newData);

    //Traverse until the last node
    while(currenthead->next != nullptr){
        currenthead = currenthead->next;
    }

    currenthead->next = newNode;
    newNode->prev = currenthead;

}

```

A new node was successfully added after the last node.

```

136 template <typename T>
137 void dllDelete(T findData, doublelist<T>** currenthead){
138     if(*currenthead == nullptr){
139         return;
140     }
141
142     doublelist<T>* currentnode= *currenthead;
143
144     while(currentnode != nullptr && currentnode-> data !=findData){
145         currentnode= currentnode->next;
146     }
147
148     if (currentnode==nullptr){
149         return;
150     }
151     if (currentnode == *currenthead){
152         *currenthead=currentnode->next;
153
154         if (currenthead !=nullptr){
155             (*currenthead)->prev= nullptr;
156         }
157
158     }else {
159         currentnode->prev->next= currentnode->next;
160         if(currentnode->next !=nullptr){
161             currentnode->next->prev= currentnode->prev;
162         }
163     }
164     delete currentnode;
165 }
166
167 #endif

```

The selected node was removed from the list.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE

main.cpp > ...

```
1  #include <iostream>
2  #include <string>
3
4
5  struct Songs{
6      std::string title;
7      Songs* next;
8      Songs* prev;
9
10     Songs(const std::string& name) : title(name), next(nullptr),prev(nullptr) {}
11 };
12
13
14 class Playlist {
15 private:
16     Songs* head;
17     Songs* current;
18
19 public:
20     Playlist() : head(nullptr),current(nullptr) {}
21     ~Playlist() {
22         if (!head) return;
23         Songs* temp = head;
24         do {
25             Songs* nextNode=temp->next;
26             delete temp;
27             temp=nextNode;
28         } while (temp !=head);
29     }
30     void insertSong(const std::string& title) {
31         Songs* newNode = new Songs(title);
32
33         if (!head) {
34             head =newNode;
35             head->next=head;
36             head->prev=head;
37             current =head;
```

```

38         std::cout<<"Added: \"" << title << "\"\n";
39         return;
40     }
41
42     Songs* tail = head->prev;
43     tail->next = newNode;
44     newNode->prev = tail;
45     newNode->next = head;
46     head->prev = newNode;
47
48     std::cout << "Added: \"" << title << "\"\n";
49 }
50
51 void removeSong(const std::string& title) {
52     if (!head) {
53         std::cout<<"Playlist is empty!\n";
54         return;
55     }
56     Songs* temp= head;
57     bool found= false;
58
59     do {
60         if (temp->title == title) {
61             found=true;
62             break;
63         }
64         temp = temp->next;
65     } while (temp != head);
66
67     if (!found) {
68         std::cout << "song \"" << title << "\" not found.\n";
69         return;
70     }

```



```

72     if (temp->next == temp) {
73         delete temp;
74         head = nullptr;
75         current = nullptr;
76     } else {
77         temp->prev->next = temp->next;
78         temp->next->prev = temp->prev;
79
80         if (temp==head) head= temp->next;
81         if (temp==current) current= temp->next;
82
83         delete temp;
84     }
85
86     std::cout << "removed: \"" << title << "\"\n";
87 }
88
89 void playAll() {
90     if (!head) {
91         std::cout << "playlist is empty!\n";
92         return;
93     }
94
95     std::cout << "\n--Playlist--\n";
96     Songs* temp = head;
97     do {
98         std::cout << "- " << temp->title;
99         if (temp==current) std::cout << " [Currently Playing]";
100        std::cout << "\n";
101        temp = temp->next;
102    } while (temp != head);
103    std::cout << "-----\n";
104 }

```

OUTPUT

```

== PLAYLIST ==
1. Add Song
2. Remove Song
3. Play All Songs
4. Next Song
5. Previous Song
6. Exit
Enter your choice (1-6): 

```

ADD SONGS

```
== PLAYLIST ==  
1. Add Song  
2. Remove Song  
3. Play All Songs  
4. Next Song  
5. Previous Song  
6. Exit  
Enter your choice (1-6): 1  
Enter song title: Lifetime  
Added: "Lifetime"
```

PLAY ALL

```
== PLAYLIST ==  
1. Add Song  
2. Remove Song  
3. Play All Songs  
4. Next Song  
5. Previous Song  
6. Exit  
Enter your choice (1-6): 3  
  
--Playlist--  
- Lifetime [Currently Playing]  
- The Apartment we wont share  
- Casual  
- Guilty as Sin?  
- Fallingforyou  
- Mean it  
-----
```

NEXT SONG

== PLAYLIST ==

1. Add Song
2. Remove Song
3. Play All Songs
4. Next Song
5. Previous Song
6. Exit

Enter your choice (1-6): 4

Now playing: The Apartment we wont share

PREVIOUS SONG

== PLAYLIST ==

1. Add Song
2. Remove Song
3. Play All Songs
4. Next Song
5. Previous Song
6. Exit

Enter your choice (1-6): 5

Now playing: Lifetime

REMOVE SONG

== PLAYLIST ==

1. Add Song
2. Remove Song
3. Play All Songs
4. Next Song
5. Previous Song
6. Exit

Enter your choice (1-6): 2

Enter song title: Fallingforyou

removed: "Fallingforyou"

```
--Playlist--
- Lifetime [Currently Playing]
- The Apartment we wont share
- Casual
- Guilty as Sin?
- Mean it
-----
```

CONCLUSION

In this activity, I learned about singly, doubly, and circular linked lists. I learned how to implement basic linked list operations like insertion, deletion, and traversal. The supplementary activity provided practical experience in implementing a circular linked list through a song playlist application. The activity was difficult, especially the procedure part because it was confusing at first but I successfully completed the activity and gained a better understanding of linked lists and their operations.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

- Code assistance
- Grammar correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

- Helped me fix my grammar into more understandable manner
- Helped in debugging and explaining what went wrong
- Suggested solutions to fix the code

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.